



Multithreading

What will be covered

OBJECT

- Thread priorities
- Use of join
- Example of join
- Need of Thread synchronization
- Synchronized code
- Interthread communication
- Methods for Interthread communication



- Thread class provides the join() method which allows one thread to wait until another thread completes its execution.
- If it is a Thread object whose thread is currently executing, then t.join() will make sure that it is terminated before the next instruction is executed by the program.
- This is a final method of the Thread class and has 2 overloaded versions. One accepts no argument whereas the other accepts time in milliseconds.


```
public static void main(String args[]){  
    Threadjoiningmethod th1=new Threadjoiningmethod ();  
    Threadjoiningmethod th2=new Threadjoiningmethod ();  
    Threadjoiningmethod th3=new Threadjoiningmethod ();  
    th1.start();  
    try{  
        th1.join();  
    }  
    catch(Exception e){  
        System.out.println(e);  
    }  
    th2.start();  
    th3.start();  
}
```

- main() thread calls join() method on th1, so main thread waits till th1 finishes.

- Here you can notice thread1 first complete its task, then thread2 and thread3 will execute.

- When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issues.
- For example, a single file is being updated by two threads. they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.
- So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time. This is implemented using a concept called monitors.

- Synchronization is a process of handling resource accessibility by multiple thread requests. The main purpose of synchronization is to avoid thread interference.

- At times when more than one thread try to access a shared resource, we need to ensure that resource will be used by only one thread at a time. The process by which this is achieved is called synchronization. The synchronization keyword in java creates a block of code referred to as critical section.

```
synchronized (object)
{
    //statement to be synchronized
}
```

- Even the methods can be declared as synchronized.

Synchronized code

```
class First
{
    synchronized public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}
```

- When a thread is executing a synchronized block, all other threads will be prevented from calling the same block. When the thread executes it completely, then only other threads are allowed

- Inter-thread communication in Java is a technique through which multiple threads communicate with each other.
- When more than one threads are executing simultaneously, sometimes they need to communicate to each other by exchanging information with each other. A thread exchanges information before or after it changes its state
- If Thread B waits for Thread A to produce data, it will waste many CPU cycles. But if threads A and B communicate with each other when they have completed their tasks, they do not have to wait and check each other's status every time.
- This type of information exchanging between threads is called inter-thread communication in Java.

- Inter thread communication in Java can be achieved by using three methods provided by Object class of java.lang package.

They are:

1. wait()
2. notify()
3. notifyAll()

- These methods can be called only from within a synchronized method or synchronized block of code otherwise, an exception named `IllegalMonitorStateException` is thrown.

- All these methods are declared as final. Since it throws a checked exception, therefore, you must be used these methods within Java try-catch block.

- `wait()` method in Java notifies the current thread to give up the monitor (lock) and to go into sleep state until another thread wakes it up by calling `notify()` method.
- The `notify()` method wakes up a single thread that called `wait()` method on the same object. If more than one thread is waiting, this method will awake one of them.
- The `notifyAll()` method is used to wake up all threads that called `wait()` method on the same object. The thread having the highest priority will run first.


```
public static void main(String[] args) throws InterruptedException {
```

```
    ThreadB b = new ThreadB();
```

```
    b.start();
```

```
    synchronized (b) {
```

```
        System.out.println("main thread calling wait() method"); // step 1
```

```
        b.wait();
```

```
        System.out.println("main thread got notification call"); // step 4
```

```
        System.out.println("total balance " + b.totalBalance);
```

```
    }
```

```
public void run() {
```

```
    synchronized (this) {
```

```
        System.out.println("child thread starts calculation for total balance"); // step 2
```

```
        for (int i = 0; i <= 50; i++) {
```

```
            totalBalance = totalBalance + i;
```

```
        }
```

```
        System.out.println("child thread gives notification call"); // step 3
```

```
        this.notify();
```

```
    }
```

```
}
```



Multithreading-II

In this chapter we will learn about need of synchronization and applying it with the help of simple synchronized block. We will even learn about interthread communication with simple methods provided in Object class. We will even see about thread deadlock senario.

Need of synchronization

1. Multiple threads are created so as CPU idle time can be avoided as much as possible. Even if one thread is halted for some reason, entire execution will not be stopped because other thread may resume and proceed with its task.
2. CPU time is utilized maximally But there is some serious drawback of creating multiple threads in the applications .
3. There may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issues.
4. For example, if multiple threads try to write within a same file then they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.
5. At times when more than one threads try to access a shared resource, we need to ensure that resource will be used by only one thread at a time. The process by which this is achieved is called synchronization.
6. Synchronization is a process of handling resource accessibility by multiple thread requests. The main purpose of synchronization is to avoid thread interference.
7. This is implemented using a concept called monitors. Each object in Java is associated with a monitor, which a thread can lock or unlock. Only one thread at a time may hold a lock on a monitor.
8. The synchronized keyword in java creates a block of code referred to as critical section.

Applying synchronization

```
synchronized (object)
{
    //statement to be synchronized
}
```

Any code written by using synchronized block or enclosed inside synchronized method will be mutually exclusive, and can only be executed by one thread at a time.

Synchronization in Java comes with the cost of performance, we need to synchronize only part of the code which absolutely

needs to be synchronized

Applying synchronization

```
synchronized (object)
{
    //statement to be synchronized
}
```

Any code written by using synchronized block or enclosed inside synchronized method will be mutually exclusive, and can only be executed by one thread at a time.

Synchronization in Java comes with the cost of performance, we need to synchronize only part of the code which absolutely needs to be synchronized.

Let us see the example :

```
//shared resource which is used by multiple threads
public class Printer
{
    PrintWriter out;
    public Printer()
    {
        out=new PrintWriter(System.out);
    }
    public void print(String header,String body,String footer)
    {
        try
        {
            out.print(header);
            out.flush();
            Thread.sleep(500);
            out.print(body);
            out.flush();
            Thread.sleep(500);
            out.print(footer);
            out.println();
            out.flush();
        }
        catch (InterruptedException e)
        {
        }
    }
}
```



```
public void print(String header,String body,String footer)
```

```
{
```

```
    try
```

```
    {
```

```
        out.print(header);
```

```
        out.flush();
```

```
        Thread.sleep(500);
```

```
        out.print(body);
```

```
        out.flush();
```

```
        Thread.sleep(500);
```

```
        out.print(footer);
```

```
        out.println();
```

```
        out.flush();
```

```
    }
```

```
    catch (InterruptedException e)
```

```
    {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

```
}
```

```
//thread which makes use of shared resource
```

```
public class PrinterApp implements Runnable
```

```
{
```

```
    private Thread t;
```

```
    Printer p;
```

```
    String header,body,footer;
```

```
    public PrinterApp(Printer p,String header,String body,String footer)
```

```
    {
```

```
        t=new Thread(this);
```

```
        this.p=p;
```

```
        this.header=header;
```

```
        this.body=body;
```

```
        this.footer=footer;
```

```
    }
```

```
    public Thread getThread()
```

```
{
```



```
public class PrinterApp implements Runnable
{
    private Thread t;
    Printer p;
    String header,body,footer;
    public PrinterApp(Printer p,String header,String body,String footer)
    {
        t=new Thread(this);
        this.p=p;
        this.header=header;
        this.body=body;
        this.footer=footer;
    }
    public Thread getThread()
    {
        return t;
    }
    public void run()
    {
        p.print(header, body, footer);
    }
}
```

//application which creates multiple threads accessing shared resource

```
public class PrinterAppDemo
{
    public static void main(String[] args)
    {
        Printer p=new Printer();
        PrinterApp app1=new PrinterApp(p, "{", "Hello World", "}");
        PrinterApp app2=new PrinterApp(p, "[", "Welcome to multithreading", "]");
        PrinterApp app3=new PrinterApp(p, "(", "Learning synchronization", ")");
        System.out.println("Printing begins.....");
        app1.getThread().start();
        app2.getThread().start();
        app3.getThread().start();
    }
}
```



```

public class PrinterAppDemo
{
    public static void main(String[] args)
    {
        Printer p=new Printer();
        PrinterApp app1=new PrinterApp(p, "{", "Hello World", "}");
        PrinterApp app2=new PrinterApp(p, "[", "Welcome to multithreading", "]");
        PrinterApp app3=new PrinterApp(p, "(", "Learning synchronization", ")");
        System.out.println("Printing begins.....");
        app1.getThread().start();
        app2.getThread().start();
        app3.getThread().start();
        try
        {
            app1.getThread().join();
            app2.getThread().join();
            app3.getThread().join();
        }
        catch (InterruptedException ex)
        {
            {}
        }
        System.out.println("Printing ends.....");
    }
}

```

Output :

```

Printing begins.....
{[ (Welcome to multithreading learning synchronization Hello World)]
Printing ends....

```

Above output is because synchronization is not applied, so printer instance is used simultaneously by multiple threads so synchronization should be applied when printer prints the data and only one thread should be able to use printer instance. So above Printer class is modified as :

```

public class Printer
{
    PrintWriter out;

```

```

public class Printer
{
    PrintWriter out;
    public Printer()
    {
        out=new PrintWriter(System.out);
    }
    public void print(String header,String body,String footer)
    {
        synchronized (this)
        {
            try
            {
                out.print(header);
                out.flush();
                Thread.sleep(500);
                out.print(body);
                out.flush();
                Thread.sleep(500);
                out.print(footer);
                out.println();
                out.flush();
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
}

```

Now the output will be
Printing begins.....

{Hello World}

(Learning synchronization)

[Welcome to multithreading]

Printing ends

Interthread Communication

1. Though normally each child thread just need to complete its own task, sometimes we may want multiple threads to work together to fulfil a task, which involves the inter-thread communication.
2. The process of testing a condition repeatedly till it becomes true is known as polling.
3. Polling is usually implemented with the help of loops to check whether a particular condition is true or not. If it is true, certain action is taken. This waste many CPU cycles and makes the implementation inefficient.

For example, in a classic queuing problem where one thread is producing data and other is consuming it.

To avoid polling, java uses three methods, namely, wait(), notify() and notifyAll().

All these methods belong to Object class as final so that all classes have them. They must be used within a synchronized block only. Otherwise it throws IllegalMonitorStateException

1. wait()-It tells the calling thread to give up the lock and go to sleep until some other thread enters the same monitor and calls notify().
2. notify()-It wakes up one single thread that called wait() on the same object. It should be noted that calling notify() does not actually give up a lock on a resource.
3. notifyAll()-It wakes up all the threads that called wait() on the same object.

Classical example of producer-consumer

//shared resource by producer and consumer thread

```
public class Stock
{
    int qtyProduced;
    int qtyConsumed;
    boolean bProduced;

    public Stock()
    {
        qtyProduced=qtyConsumed=0;
        bProduced=false;
    }

    public void produce()
    {
        while(bProduced) //thread polling checking whether already produced
```

```
public class Stock
{
    int qtyProduced;
    int qtyConsumed;
    boolean bProduced;
    public Stock()
    {
        qtyProduced=qtyConsumed=0;
        bProduced=false;
    }
    public void produce()
    {
        while(bProduced) //thread polling checking whether already produced
        ;
        qtyProduced++;
        bProduced=true;
        System.out.println("Produced : "+qtyProduced);
    }
    public void consume()
    {
        while(!bProduced) //thread polling checking whether already consumed
        ;
        qtyConsumed++;
        bProduced=false;
        System.out.println("Consumed : "+qtyConsumed);
    }
}
//producer thread
public class Producer implements Runnable
```



```
//producer thread
public class Producer implements Runnable
{
    Thread t;
    Stock s;
    boolean bRun;
    public Producer(Stock s)
    {
        t=new Thread(this);
        this.s=s;
        bRun=true;
    }
    public Thread getThread()
    {
        return t;
    }
    public void run()
    {
        while(bRun)
            s.produce();
    }
    public void stopNow()
    {
        bRun=false;
    }
}

//consumer thread
public class Consumer implements Runnable
{
    Thread t;
    Stock s;
```

```
//consumer thread
```

```
public class Consumer implements Runnable
```

```
{
```

```
    Thread t;
```

```
    Stock s;
```

```
    boolean bRun;
```

```
    public Consumer(Stock s)
```

```
    {
```

```
        t=new Thread(this);
```

```
        this.s=s;
```

```
        bRun=true;
```

```
    }
```

```
    public Thread getThread()
```

```
    {
```

```
        return t;
```

```
    }
```

```
    public void run()
```

```
    {
```

```
        while(bRun)
```

```
            s.consume();
```

```
    }
```

```
    public void stopNow()
```

```
    {
```

```
        bRun=false;
```

```
    }
```

```
}
```

```
//application class
```

```
public class CommDemo {
```

```
    public static void main(String[] args) throws InterruptedException
```

```
    {
```

```
        Stock s=new Stock();
```



```
//application class
public class CommDemo {
    public static void main(String[] args) throws InterruptedException
    {
        Stock s=new Stock();
        Producer p=new Producer(s);
        Consumer c=new Consumer(s);
        p.getThread().start();
        c.getThread().start();
        System.out.println("Demo begins.....");
        Thread.sleep(5000);
        p.stopNow();
        c.stopNow();
        p.getThread().join();
        c.getThread().join();
        System.out.println("Demo ends.....");
        System.out.println("Total qty produced : "+s.qtyProduced);
        System.out.println("Total qty consumed : "+s.qtyConsumed);
    }
}
```

Producer thread checks if some quantity is already produced, it waits till it gets notified by consumer thread.

Note : Methods are called from synchronized context and throw InterruptedException.

Difference between sleep and wait

Method called on

wait() – Call on an object; current thread must synchronize on the lock object.

sleep() – Call on a Thread; always currently executing thread.

Synchronized

wait() – when synchronized multiple threads access same Object one by one.

sleep() – when synchronized multiple threads wait for sleep over of sleeping thread.

lock duration

Difference between sleep and wait

Method called on

wait() – Call on an object; current thread must synchronize on the lock object.

sleep() – Call on a Thread; always currently executing thread.

Synchronized

wait() – when synchronized multiple threads access same Object one by one.

sleep() – when synchronized multiple threads wait for sleep over of sleeping thread.

Lock duration

wait() – release the lock for other objects to have chance to execute.

sleep() – keep lock for at least t times if timeout specified or somebody interrupt.

Wake up condition

wait() – until call notify(), notifyAll() from object

sleep() – until at least time expire or call interrupt().

Usage

sleep() – for time-synchronization

wait() – for multi-thread-synchronization.

Thread Deadlock

Synchronization is used to make the class or method thread-safe which means only one thread can have lock of synchronized method and use it, other threads have to wait till the lock releases and anyone of them acquire that lock. But sometimes it also causes a problem which is called Deadlock.

Deadlock in java is a programming situation where two or more threads are blocked forever. Java deadlock situation arises with at least two threads and two or more resources.

A deadlock occurs when two or more threads wait forever for a lock or resource held by another of the threads. Consequently, an application may stall or fail as the deadlocked threads cannot progress.

Deadlock is a situation of complete Lock, when no thread can complete its execution because lack of resources. Thread1 holds the resource1 and waits for resource2 and Thread2 holds resource2 and waits for resource1.

```
public static void main(String[] args)
{
```

```
    final Pen pn = new Pen();
```



```
public static void main(String[] args)
{
    final Pen pn = new Pen();
    final Paper pr = new Paper();
    Thread t1 = new Thread() {
        public void run()
        {
            synchronized(pn)
            {
                System.out.println("Thread1 is holding Pen");
                try{
                    Thread.sleep(1000);
                }
                catch(InterruptedException e){
                    // do something
                }
                synchronized(pr)
                {
                    System.out.println("Requesting for Paper");
                }
            }
        }
    };
    Thread t2 = new Thread() {
        public void run()
        {
            synchronized(pr)
            {
                System.out.println("Thread2 is holding Paper");
                try{
                    Thread.sleep(1000);
                }
                catch(InterruptedException e){
                    // do something
                }
                synchronized(pn)
                {
                    System.out.println("Requesting for Pen");
                }
            }
        }
    };
    t1.start();
    t2.start();
}
```

```

Thread t2 = new Thread() {
    public void run()
    {
        synchronized(pr)
        {
            System.out.println("Thread2 is holding Paper");
            try {
                Thread.sleep(1000);
            }
            catch (InterruptedException e) {
                // do something
            }
            synchronized(pn)
            {
                System.out.println("requesting for Pen");
            }
        }
    }
};
t1.start();
t2.start();
}
}

```

Assignments

1. Create thread for printing data on the console with the help of Printer object. Data is simply 3 different strings. Printer object provides print() method to display data on the console. Use sleep() method of the thread if data is very small. Create atleast 3 instances of the thread in main() method. Observe the outcome. Apply synchronization and observe change in the outcome.
2. Create a thread which will find the divisors of a number and print all the divisors in a file. Create 3 instances of a thread running from main thread which will use the same file for writing list of all divisors. Use synchronization so that only one thread will use the file at a time. e.g. Divisors of number 32 are : 1,2,4,8,16,32
3. Create producer and consumer demo for implementing inter-thread communication.

Important Questions for: Multithreading-II


```

System.out.println("Thread2 is holding Paper");
try {
    Thread.sleep(1000);
}
catch (InterruptedException e) {
    // do something
}
synchronized (pn)
{
    System.out.println("requesting for Pen");
}
}
};
t1.start();
t2.start();
}
}

```

Assignments

1. Create thread for printing data on the console with the help of Printer object. Data is simply 3 different strings. Printer object provides print() method to display data on the console. Use sleep() method of the thread if data is very small. Create atleast 3 instances of the thread in main() method. Observe the outcome. Apply synchronization and observe change in the outcome.
2. Create a thread which will find the divisors of a number and print all the divisors in a file. Create 3 instances of a thread running from main thread which will use the same file for writing list of all divisors. Use synchronization so that only one thread will use the file at a time. e.g. Divisors of number 32 are : 1,2,4,8,16,32
3. Create producer and consumer demo for implementing inter thread communication.

Important Questions for: Multithreading-II

Interview Questions

Write syntax for multithreading (on pen and paper). Write using both runnable and thread class. I said I don't exactly know the syntax for runnable I can write for thread class. They said try writing for both regardless.